
Rapport de 1^{ère} année d'apprentissage

Développement et modernisation de la plateforme SALOME



Nicolas SAIKALY

Spécialité : Informatique et ingénierie mathématique
Polytech Paris-Saclay

Entreprise d'accueil : CEA Saclay – SGLS/LESIM

Encadré par : Christophe BOURCIER
Mohd Afeef BADRI

Année académique : 2025 – 2026

Remerciements

Je tiens à remercier en premier lieu mon chef de service, Dr. Erwan ADAM, et mon chef de laboratoire, M. Nicolas CROUZET, pour la confiance qu'ils m'ont accordée en m'accueillant au sein du CEA.

Je remercie mes deux maîtres d'apprentissage, M. Christophe BOURCIER et Dr. Mohd Afeef BADRI, pour leur encadrement tout au long de cette première année. Leur grande disponibilité, leur pédagogie et leurs conseils techniques m'ont permis de progresser rapidement et de mener à bien les missions qui m'ont été confiées.

Je remercie également l'ensemble de l'équipe du laboratoire LESIM pour son accueil chaleureux et sa bienveillance. Cette excellente ambiance de travail a rendu cette première expérience industrielle particulièrement enrichissante et a grandement facilité mon intégration.

Je tiens également à remercier l'équipe pédagogique de Polytech Paris-Saclay et de l'Université Paris-Saclay pour son accompagnement et la qualité de la formation dispensée tout au long de cette année en alternance. J'exprime ma gratitude tout particulièrement envers mon tuteur académique, Pr. Marc BABOULIN, pour son suivi attentif et ses conseils précieux qui ont favorisé le bon déroulement de cette année entre l'école et l'entreprise.

Enfin, je remercie ma famille pour son soutien constant tout au long de cette première année.

Résumé

Ce rapport présente le bilan de ma première année d'apprentissage au sein du Laboratoire Environnement de Simulation (LESIM) du CEA Paris-Saclay, effectuée dans le cadre de ma formation d'ingénieur en informatique et ingénierie mathématique à Polytech Paris-Saclay.

Mes travaux ont porté sur le développement et la modernisation de deux plugins de maillage pour la plateforme de simulation numérique SALOME. Le premier, **MeshBooleanPlugin**, permet d'effectuer des opérations booléennes entre maillages. J'ai notamment implémenté un mécanisme d'annulation de calcul multiplateforme (Linux/Windows), extrait une API Python autonome, sécurisé la gestion des ressources système, et intégré le plugin au système de **Dump Study** de SALOME. Le second plugin, **PolyMeshPlugin**, transforme un maillage quelconque en maillage polyédrique utilisant trois algorithmes distincts. J'ai refactorisé l'ensemble du pipeline pour garantir sa portabilité, ajouté un suivi d'exécution en temps réel, et développé un exécutable C++ dédié nommé **geogram2med** s'appuyant sur la bibliothèque **MEDCoupling** pour la conversion directe des données.

Ces contributions, validées par des tests automatisés, ont abouti à des **pull requests** officiellement intégrées au code source de SALOME et qui seront donc disponibles en open-source dans les prochaines versions de la plateforme.

Mots clés : maillage, simulation numérique, SALOME, opérations booléennes, maillage polyédrique, **MEDCoupling**, Python, C++, CEA.

Abstract

This report presents an overview of my first apprenticeship year at the Simulation Environment Laboratory (LESIM) of CEA Paris-Saclay, carried out as part of my engineering degree in Computer Science and Applied Mathematics at Polytech Paris-Saclay.

My work focused on the development and modernization of two meshing plugins for SALOME, the numerical simulation platform, co-developed by CEA and EDF. The first plugin, **Mesh-BooleanPlugin**, performs boolean operations between meshes. My contributions include implementing a cross-platform computation cancellation mechanism (Linux/Windows), extracting a standalone Python API, securing system resource management, and integrating the plugin into SALOME's **Dump Study** system. The second plugin, **PolyMeshPlugin**, converts arbitrary meshes into polyhedral meshes using three distinct algorithms. I refactored the entire pipeline to ensure cross-platform portability, added real-time execution tracking, and developed a dedicated C++ executable called **geogram2med**, which relies on the **MEDCoupling** library for direct data conversion.

These contributions, validated by automated testing, resulted in **pull requests** that have been officially merged into the SALOME main branch and will be available as open-source in the upcoming releases of the platform.

Keywords : meshing, numerical simulation, SALOME, boolean operations, polyhedral meshing, MEDCoupling, Python, C++, CEA.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Introduction	1
1 Environnement technologique : SALOME et l'enjeu du maillage	3
1.1 La plateforme SALOME	3
1.2 Le module SMESH et le système de plugins	4
2 Premier trimestre : MeshBooleanPlugin	6
2.1 Présentation du plugin et état initial	6
2.2 Implémentation du bouton Cancel	7
2.2.1 Problématique et contrainte multiplateforme	7
2.2.2 Solution retenue : subprocess et QThread	7
2.2.3 Intégration au workflow Git et résolution de conflits	8
2.3 Extraction de l'API Python et découplage GUI	8
2.3.1 Motivation architecturale	8
2.3.2 Refactorisation et API	8
2.3.3 Gestion des ressources et qualité de code	8
2.3.4 Mode verbose et facilitation du débogage	9
2.4 Intégration au Dump Study	9
2.4.1 Fonctionnement du Dump Study dans SALOME	9
2.4.2 Problème identifié	9
2.4.3 Solution implémentée	9
2.5 Tests et validation	10
2.6 Cas d'usage scientifiques et perspectives	11
3 Deuxième trimestre : PolyMeshPlugin	13
3.1 Présentation du plugin et état initial	13
3.2 Intérêt du maillage polyédrique	14
3.3 État du plugin à la reprise	14
3.4 Refactorisation multiplateforme	15
3.4.1 Remplacement de pexpect par subprocess	15
3.4.2 Barre de progression en temps réel	15
3.4.3 Gestion des erreurs et robustesse	15
3.5 Refonte architecturale du pipeline Geogram	16
3.5.1 Problème initial	16
3.5.2 Conception et développement de geogram2med	17
3.5.3 Résultats et bénéfices	18
3.6 Cas test et validation	18
4 Responsabilité sociétale et transition écologique au CEA	19
4.1 Contexte et enjeux DDRS	19
4.2 La mission énergétique du CEA comme réponse aux enjeux climatiques	19
4.3 L'open source comme pratique de numérique responsable	19

4.4	Regard critique sur les pratiques observées	20
4.4.1	Points forts	20
4.4.2	Points d'amélioration et limites observées	20
4.5	Lien entre mes missions et les enjeux DDRS	20
5	Bilan et retour d'expérience	21
5.1	Compétences techniques acquises	21
5.1.1	Python et architecture logicielle	21
5.1.2	C++ et MEDCoupling	21
5.1.3	Git et workflow collaboratif	21
5.1.4	Formation et montée en compétence sur SALOME	21
5.1.5	Environnement Linux	22
5.1.6	Veille technologique et autonomie d'apprentissage	22
5.2	Compétences transversales et humaines	22
5.2.1	Autonomie et organisation	22
5.2.2	Communication technique	22
5.2.3	Intégration dans un environnement professionnel	22
5.3	Difficultés rencontrées	23
5.3.1	La contrainte cross-platform	23
5.3.2	Prendre en main un code existant	23
5.3.3	Équilibre école-entreprise	23
5.4	Perspectives pour les années 2 et 3	23
	Conclusion	25
	Bibliographie	26
	Lexique	27

Table des figures

1	Organisation temporelle des travaux réalisés au cours de la première année d'apprentissage (septembre 2025 – avril 2026).	2
1.1	Architecture modulaire de SALOME et place des plugins développés au sein du module SMESH.	4
1.2	Interface utilisateur du module SMESH de SALOME : vue scindée montrant la géométrie à gauche et le maillage correspondant à droite.	4
1.3	Comparaison entre le maillage tétraédrique d'entrée et le maillage polyédrique généré par le PolyMeshPlugin sur la géométrie du coude de tuyauterie.	5
2.1	Les trois opérations booléennes du MeshBooleanPlugin appliquées à deux maillages triangulaires partiellement superposés.	6
2.2	Les deux états de la boîte de dialogue du MeshBooleanPlugin . Le bouton Cancel ne devient cliquable qu'une fois le calcul lancé, évitant toute interaction non intentionnelle.	8
2.3	Extrait d'un script Python généré par le dump study de SALOME après une session incluant trois opérations booléennes. Les appels à l'API sont propres et directement rejouables sans interaction manuelle.	10
2.4	Résultat de l'exécution des 15 sous-tests automatisés du MeshBooleanPlugin en ligne de commande, couvrant les 5 algorithmes disponibles et les trois opérations booléennes.	11
2.5	Exemple de microstructure poreuse d'électrode de batterie lithium-ion. Ce type de géométrie complexe est un cas d'usage cible pour les opérations booléennes entre maillages dans le cadre des recherches sur le stockage d'énergie au CEA. . .	11
2.6	Modélisation du béton et de ses agrégats : configuration géométrique cible pour une application des opérations booléennes entre maillages dans le cadre de simulations de résistance mécanique de matériaux composites. Images générées avec SALOME.	12
3.1	Comparaison des trois types de maillages polyédriques générés par le PolyMeshPlugin sur une même géométrie d'entrée. Chaque algorithme produit une topologie cellulaire différente. Le rendu bleu correspond au maillage surfacique, tandis que le rendu rose présente le maillage volumique en vue éclatée et en coupe. . . .	13
3.2	Comparaison quantitative entre maillages tétraédrique, hexaédrique et polyédrique sur un cas de conduite simple. Source : [11].	14
3.3	Boîte de progression affichée pendant le calcul du PolyMeshPlugin, mise à jour en temps réel par lecture caractère par caractère de la sortie du processus. . . .	15
3.4	Boîte de dialogue d'avertissement affichée par le PolyMeshPlugin lorsque cfMesh génère un maillage vide. Le message guide l'utilisateur vers le paramètre à ajuster. .	16
3.5	Refonte du pipeline de conversion Geogram : suppression du format intermédiaire .ovm et du parsing Python fragile au profit d'une conversion C++ directe vers .med, s'appuyant sur les bibliothèques Geogram et MEDCoupling. Les boîtes jaunes représentent les fichiers, les boîtes colorées les algorithmes de conversion. .	17
3.6	Validation du PolyMeshPlugin sur une pièce mécanique. La vue superposée (à droite) démontre que l'algorithme Geogram avec export de seeds conserve parfaitement le raffinement local : chaque point seed du maillage tétraédrique devient le centre d'une cellule polyédrique.	18

Introduction

La simulation numérique occupe aujourd’hui une place centrale dans l’ingénierie scientifique moderne. Qu’il s’agisse de modéliser le comportement d’un réacteur nucléaire, de simuler des écoulements fluides complexes ou d’étudier la résistance mécanique de matériaux composites, ces méthodes permettent de remplacer des expériences physiques coûteuses, dangereuses ou tout simplement impossibles à réaliser à grande échelle. Au cœur de ces workflows de simulation se trouve une étape critique et souvent sous-estimée : la génération et la manipulation du maillage, qui conditionne directement la précision et la performance des calculs numériques.

C’est dans ce contexte que s’inscrit mon apprentissage au sein du CEA, l’un des acteurs majeurs de la recherche publique française. Fondé en 1945 par le général De Gaulle dans le contexte de l’après-guerre pour développer les applications civiles et militaires de l’énergie nucléaire, le CEA compte aujourd’hui plus de 21 000 salariés répartis sur neuf centres en France, pour un budget annuel de 5,8 milliards d’euros. Il se distingue comme **premier organisme public de recherche français déposant de brevets**, avec 598 demandes déposées auprès de l’INPI (Institut National de la Propriété Industrielle) en 2024, figurant ainsi dans le top 5 national toutes catégories confondues aux côtés des plus grands groupes industriels français. Le centre CEA Paris-Saclay, où j’effectue mon apprentissage, rassemble à lui seul près de 7 000 personnes et constitue le plus grand des neuf centres du CEA. Il est **organisme partenaire et membre fondateur de l’Université Paris-Saclay**, qui représente 13 % de la recherche française et figure au 13^e rang mondial dans le classement de Shanghai. Ce lien étroit entre le CEA et l’Université Paris-Saclay est au fondement même de mon contrat d’apprentissage, qui associe ma formation d’ingénieur à Polytech Paris-Saclay à une immersion professionnelle dans un environnement de recherche d’excellence.

Avant de présenter mes travaux, il convient de situer brièvement mon parcours professionnel. Après deux années de classes préparatoires en mathématiques et physique, j’ai intégré Polytech Paris-Saclay pour le début de mon cycle ingénieur en alternance, en Informatique et Ingénierie Mathématique. Cette filière, à la croisée de l’informatique et des mathématiques appliquées, m’a permis de développer des bases solides en algorithmique, en programmation C++, en calcul numérique et en architecture logicielle - des compétences directement mobilisées au quotidien dans mes missions au CEA. C’est dans ce cadre que j’ai choisi de réaliser mes trois années d’école d’ingénieur en alternance, sous contrat d’apprentissage. Ce format m’a semblé être la meilleure façon d’ancrer ma formation dans une pratique professionnelle réelle dès le début du cycle ingénieur, plutôt que d’attendre un stage de fin d’études pour confronter mes connaissances académiques à un environnement industriel.

Mon accueil se fait plus précisément au sein du Laboratoire Environnement de Simulation (LESIM), rattaché au Service de Génie Logiciel (SGLS) de la Direction des Energies (DES). J’y suis intégré comme apprenti ingénieur logiciel, avec des missions propres confiées par mes deux maîtres d’apprentissage. Ce laboratoire a pour mission le développement et la maintenance de plateformes logicielles scientifiques open source, au premier rang desquelles figure **SALOME**¹ [1] — une plateforme de simulation numérique co-développée par le CEA et EDF depuis l’an 2000, publiée en open source, et utilisée dans des domaines aussi variés que l’énergie, l’aéronautique, la marine et l’automobile. Le laboratoire développe également des outils pour la mise en données et le post-traitement de plusieurs codes de calcul du CEA, s’appuyant majoritairement sur l’architecture SALOME. Il participe également à des projets nationaux et européens sur les codes de calculs pour la simulation des matériaux.

Dans ce cadre, mes travaux de première année ont porté sur le développement et la modernisation de deux plugins du module de maillage de SALOME, appelé SMESH : le **MeshBoo-**

leanPlugin [2], qui permet d'effectuer des opérations booléennes entre maillages surfaciques, et le **PolyMeshPlugin** [3], qui transforme un maillage surfacique ou volumique en maillage polyédrique via trois algorithmes distincts. Ces deux plugins s'adressent à des besoins concrets en simulation numérique, notamment lorsqu'aucun modèle CAO² n'est disponible ou que des géométries complexes doivent être traitées directement à partir de données brutes telles que des images issues de tomographie³ — comme par exemple la reconstruction poreuse d'électrodes de batteries ou de conduites industrielles. Le **PolyMeshPlugin** répond quant à lui à un besoin de réduction des temps de calcul en CFD, en produisant des maillages polyédriques qui convergent plus rapidement que leurs équivalents tétraédriques pour un même niveau de précision.

La figure 1 présente l'organisation temporelle des travaux réalisés au cours de cette première année.

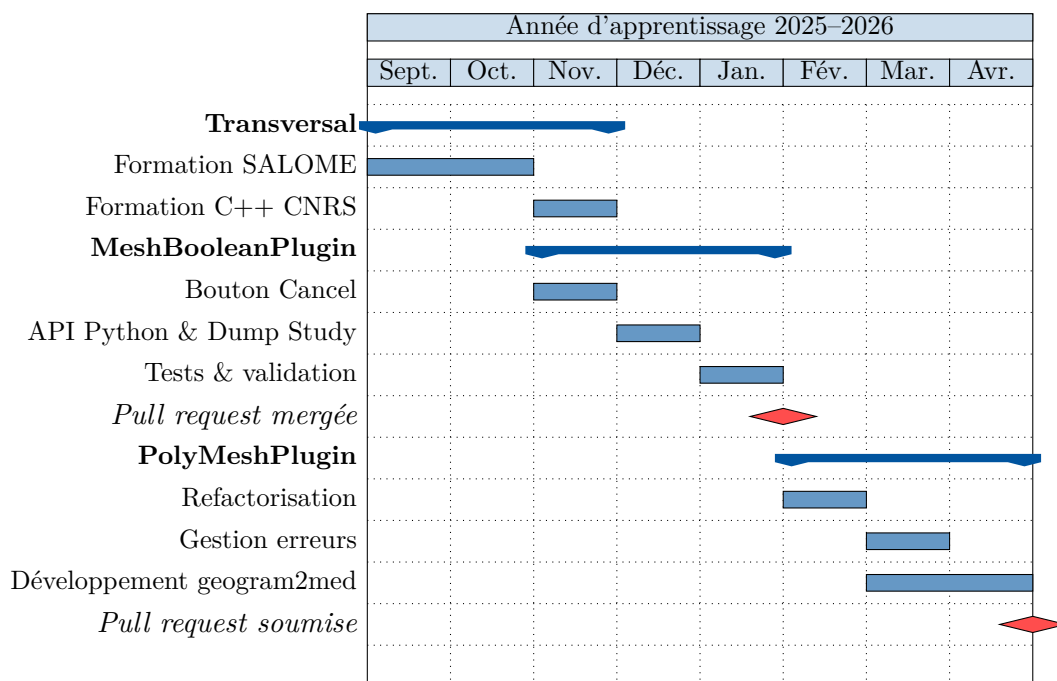


FIGURE 1 – Organisation temporelle des travaux réalisés au cours de la première année d'apprentissage (septembre 2025 – avril 2026).

Cette organisation reflète deux phases bien distinctes. La première, transversale, a été consacrée à la montée en compétence sur l'environnement SALOME et les outils du laboratoire, avec notamment une formation sur la plateforme et une formation C++ scientifique de 21 heures dispensée par le CNRS. Les missions techniques ont ensuite occupé chacune environ deux mois, un rythme qui intègre les périodes scolaires réduisant le temps effectif en entreprise à 16 semaines sur 8 mois. Ce découpage clair entre montée en compétence et contribution active a permis d'aborder chaque plugin avec suffisamment de recul technique pour proposer des solutions architecturales solides plutôt que de simples correctifs.

Ce rapport présente l'ensemble des travaux réalisés au cours de cette première année d'apprentissage, de septembre 2025 à avril 2026, ce qui représente 16 semaines de travail effectif. Après une présentation du CEA et de mon environnement de travail, nous détaillerons les contributions techniques apportées à chacun des deux plugins, avant d'aborder les enjeux de responsabilité sociétale liés à mon activité et les perspectives pour la suite de ce contrat de trois ans.

1 | Environnement technologique : SALOME et l'enjeu du maillage

1.1 La plateforme SALOME

Initiée en 2000 par un partenariat stratégique entre le CEA et EDF, SALOME est une plateforme logicielle *open source* dédiée à la simulation numérique. Son objectif est de fournir un environnement générique complet, allant de la conception assistée par ordinateur (CAO) jusqu'au post-traitement des résultats, en passant par le couplage de solveurs physiques.

Travailler sur SALOME implique de s'insérer dans un projet d'une envergure logicielle massive. L'architecture de la plateforme repose sur :

- o **Une base de code** : SALOME est une plateforme modulaire développée principalement en C++ (pour les calculs intensifs et la gestion de la mémoire) et Python (pour le prototypage, l'interface et l'automatisation). Le cœur de la plateforme est estimé à **quelques millions de lignes de code** tandis que l'ensemble de l'écosystème (incluant les pré-requis) dépasse **les dizaines de millions de lignes de code**.
- o **Un rayonnement international** : Bien que le nombre exact d'utilisateurs soit complexe à recenser, les statistiques de téléchargement confirment la large diffusion internationale de SALOME, avec un volume dépassant 5000 téléchargements mensuels. La cartographie de leur répartition mondiale témoigne d'une présence sur tous les continents, avec une majorité sur le territoire français. On estime l'utilisation en interne au CEA et EDF à près de 1000 utilisateurs. SALOME est accessible via son site officiel [1] et son code source est hébergé sur GitHub [4]. La plateforme est aussi utilisée mondialement par des chercheurs et des ingénieurs dans des secteurs critiques (nucléaire, aéronautique, automobile). Sachant que les téléchargements ne reflètent pas entièrement l'usage réel, SALOME étant largement déployé via des installations internes dans les environnements industriels et les clusters de calcul. Au-delà de la plateforme elle-même, SALOME constitue une brique logicielle fondatrice pour le CEA et EDF pour la construction d'applications métiers spécifiques, comprenant en général SALOME associé à un ou plusieurs codes de calcul et leur interface graphique dédiée. La plateforme suit un rythme de publication soutenu de **deux versions majeures par an**, témoignant de la vitalité et de la maturité du projet.
- o **Une exigence de qualité stricte** : Le code est soumis à une intégration continue rigoureuse comprenant des milliers de tests (unitaires et d'intégration) pour garantir la non-régression à chaque nouvelle version.

L'architecture de SALOME repose sur de nombreux modules qui gèrent chacun une étape spécifique de la simulation (par exemple **GEOM** pour la CAO, **SMESH** pour le maillage ou **ParaViS** pour la visualisation). Cette modularité permet aux développeurs d'isoler les responsabilités logicielles et de contribuer à un module sans impacter les autres. La figure 1.1 illustre cette organisation et situe le périmètre de mes travaux au sein de l'écosystème.

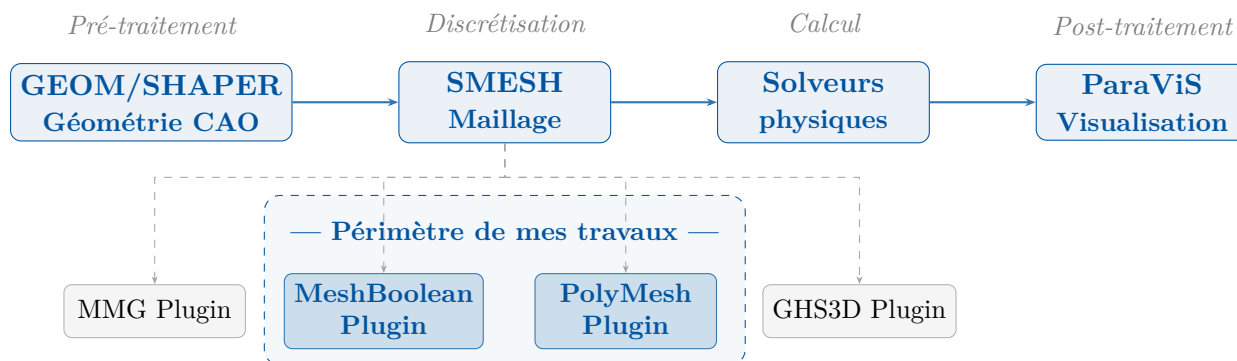


FIGURE 1.1 – Architecture modulaire de SALOME et place des plugins développés au sein du module SMESH.

L'interface principale du module SMESH, illustrée par la figure 1.2, permet de visualiser et de manipuler ces géométries discrétisées.

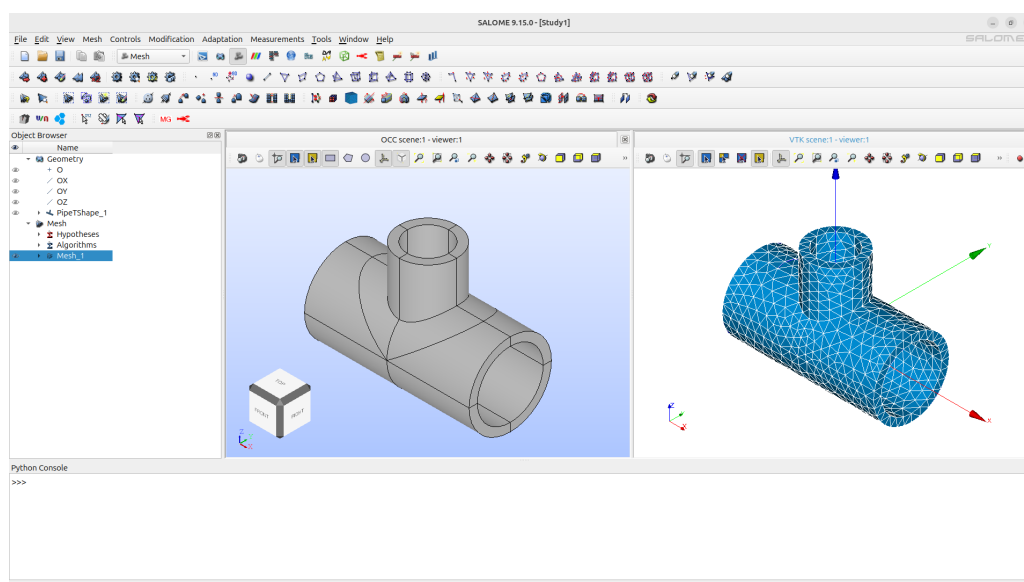


FIGURE 1.2 – Interface utilisateur du module SMESH de SALOME : vue scindée montrant la géométrie à gauche et le maillage correspondant à droite.

1.2 Le module SMESH et le système de plugins

Au cours de mon apprentissage, j'ai exclusivement travaillé sur le module **SMESH**. Ce module est le cœur de la plateforme dédié à la génération, l'édition et la manipulation de maillages.

Le maillage est l'opération fondamentale qui consiste à discrétiser un domaine géométrique continu en un ensemble fini d'éléments — en 3D : tétraèdres, hexaèdres, polyèdres ; en 2D : triangles, quadrangles, polygones. Ce sont sur ces éléments que les solveurs physiques vont calculer les variables d'intérêt (pression, température, etc.). Une erreur ou une approximation lors de cette étape compromet l'intégralité de la simulation en aval. La figure 1.3 illustre la différence visuelle entre une approche de maillage tétraédrique et l'approche polyédrique sur laquelle se concentrent mes développements.

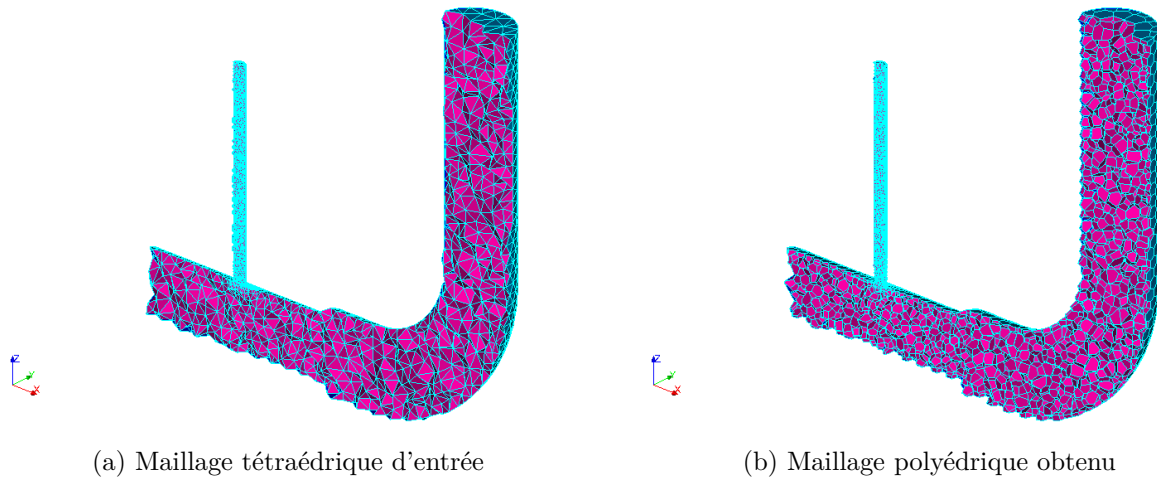


FIGURE 1.3 – Comparaison entre le maillage tétraédrique d'entrée et le maillage polyédrique généré par le PolyMeshPlugin sur la géométrie du coude de tuyauterie.

Pour répondre à des besoins très spécifiques (comme l'édition de maillages sans CAO ou la conversion de formats), le module SMESH est extensible via un système de **plugins**. Un plugin dans SALOME est un module additionnel qui vient se greffer à l'interface principale pour offrir de nouvelles fonctionnalités. C'est précisément sur le développement et la refonte architecturale de deux de ces plugins sur lesquels se sont concentrées mes missions de première année.

2 | Premier trimestre : MeshBooleanPlugin

2.1 Présentation du plugin et état initial

Le **MeshBooleanPlugin** est un plugin du module SMESH de SALOME permettant d'effectuer des **opérations booléennes entre deux maillages volumiques**. Une opération booléenne sur des maillages consiste à calculer, à partir de deux surfaces discrétisées en entrée, un nouveau maillage correspondant à leur union, leur intersection ou leur différence. Ces opérations sont fondamentales dans les workflows de simulation numérique, notamment lorsqu'aucun modèle CAO n'est disponible en amont - par exemple dans le cas de données issues de la tomographie, où le maillage est directement reconstruit à partir d'images scanner sans passer par une étape de modélisation géométrique.

Le plugin intègre **six algorithmes de calcul distincts**, chacun correspondant à une bibliothèque C++ spécialisée dans les opérations booléennes sur maillages : CGAL [5], CORK [6], MCUT [7], libigl [8], IRMB [9] et VTK [10]. Ce choix multiple permet à l'utilisateur de sélectionner l'algorithme le mieux adapté à sa géométrie et à ses contraintes de robustesse, certains algorithmes fonctionnent là où les autres échouent.

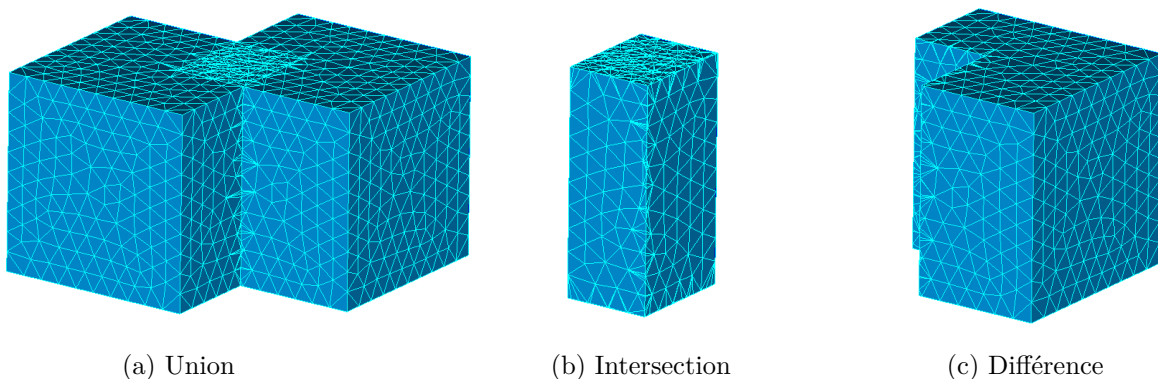


FIGURE 2.1 – Les trois opérations booléennes du **MeshBooleanPlugin** appliquées à deux maillages triangulaires partiellement superposés.

Il est à noter que parmi ces trois opérations, la **différence** est la seule qui n'est pas commutative : le résultat dépend de l'ordre dans lequel les deux maillages sont sélectionnés. Soustraire le maillage B du maillage A produit un résultat différent de soustraire A de B, contrairement à l'union et à l'intersection qui sont indépendantes de l'ordre de sélection.

Lorsque j'ai pris ce plugin en charge, en novembre 2025, il présentait plusieurs lacunes importantes qui limitaient son utilisabilité en conditions réelles :

- o Le plugin était accessible uniquement via un sous-menu du module SMESH, sans entrée visible dans la barre d'outils, ce qui le rendait difficile à découvrir pour un nouvel utilisateur.
- o Une fois un calcul lancé, **il était impossible de l'interrompre** : l'utilisateur devait soit attendre la fin de l'exécution, soit terminer le processus SALOME entièrement, perdant ainsi toutes les données non sauvegardées.
- o La logique algorithmique et le code de l'interface graphique étaient entremêlés dans le code Python, rendant ainsi toute maintenance ou évolution difficile.

- o Le plugin n'avait aucune API⁴ Python exposée, empêchant toute utilisation dans des scripts d'automatisation.
- o Aucun mécanisme de débogage n'était disponible pour diagnostiquer les erreurs en production.

Mes travaux sur ce plugin se sont étalés sur environ **quatre à cinq semaines**, de novembre 2025 à janvier 2026, et ont abouti à une *pull request* officiellement mergée dans la branche principale du dépôt GitHub de SALOME.

2.2 Implémentation du bouton Cancel

2.2.1 Problématique et contrainte multiplateforme

La première tâche concrète qui m'a été confiée consistait à permettre à l'utilisateur d'interrompre un calcul en cours. Ce besoin, en apparence simple, s'est révélé être un premier défi technique réel, notamment en raison de la contrainte fondamentale de SALOME : la plateforme doit fonctionner de manière **identique sur Linux et sur Windows**.

Une première approche, basée sur le module Python `multiprocessing` qui permet de lancer l'algorithme sous forme de processus séparé, m'a permis d'obtenir rapidement un résultat fonctionnel sur Linux : l'algorithme était lancé dans un processus fils, et un signal de terminaison lui était envoyé lors du clic sur le bouton d'annulation. Cependant, lors des tests sur Windows, cette solution s'est révélée inopérante. La gestion des processus sous Windows diffère fondamentalement de celle de Linux : les mécanismes de *fork* ne sont pas disponibles de la même manière, et l'envoi de signaux entre processus suit des règles différentes. C'est ma première leçon concrète de l'année : **ce qui fonctionne sur Linux ne fonctionne pas nécessairement sur Windows**, et toute solution doit être validée sur les deux systèmes.

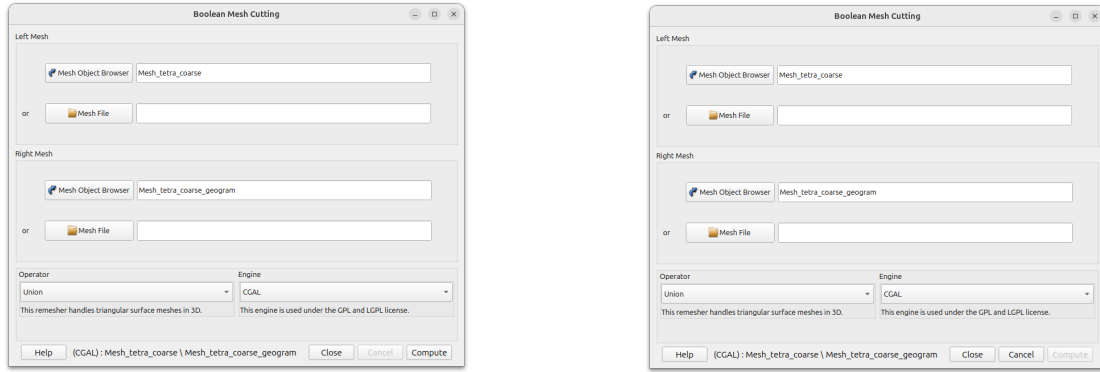
2.2.2 Solution retenue : subprocess et QThread

Après analyse et recherche, j'ai opté pour une architecture combinant deux composants de la bibliothèque standard Python et du framework Qt :

- o **subprocess** : le module standard Python permettant de lancer un processus externe de manière portable. L'algorithme de calcul est invoqué comme un sous-processus indépendant, ce qui l'isole du processus principal de SALOME.
- o **QThread** : un thread de surveillance Qt qui monitore l'état du sous-processus de manière asynchrone, sans bloquer l'interface graphique. Lors du clic sur le bouton d'annulation, un signal Qt est émis vers ce thread, qui se charge de terminer proprement le sous-processus.

Cette architecture présente deux avantages majeurs. D'une part, elle repose entièrement sur des composants standards déjà présents dans l'environnement SALOME, sans introduire de dépendance externe. D'autre part, le comportement obtenu est rigoureusement identique sur Linux et sur Windows, ce qui satisfait la contrainte de portabilité.

Un soin particulier a été apporté à l'expérience utilisateur : le bouton Cancel est grisé et inactif tant qu'aucun calcul n'est en cours. Dès que l'utilisateur lance une opération via le bouton Compute, celui-ci se grise à son tour tandis que le bouton Cancel devient actif, permettant d'interrompre proprement le processus. La figure 2.2 illustre ces deux états de l'interface.



(a) État initial : Compute actif, Cancel grisé.

(b) Lors du calcul : Cancel actif, Compute grisé.

FIGURE 2.2 – Les deux états de la boîte de dialogue du **MeshBooleanPlugin**. Le bouton Cancel ne devient cliquable qu’une fois le calcul lancé, évitant toute interaction non intentionnelle.

2.2.3 Intégration au workflow Git et résolution de conflits

Cette contribution a été soumise sous forme de *pull request*⁵ sur le dépôt GitHub officiel du plugin. L’intégration a impliqué une étape de résolution de conflits non triviale : pendant le développement de ma branche, l’équipe de développement de SALOME avait modifié l’ensemble des imports du projet pour les faire passer par `salome.kernel`. J’ai résolu ce conflit en utilisant un **rebase interactif Git**, ce qui m’a permis de rejouer proprement mes commits par-dessus les modifications de la branche principale sans altérer l’historique du projet. La pull request a été revue, validée et **mergée avec succès** par mes encadrants.

2.3 Extraction de l’API Python et découplage GUI

2.3.1 Motivation architecturale

SALOME peut être utilisé de deux manières distinctes : via son interface graphique interactive, ou via des **scripts Python** exécutés en mode console. Pour qu’un plugin soit pleinement intégré à la plateforme, il doit pouvoir être utilisé dans les deux contextes. Or, dans l’état initial du plugin, toute la logique de calcul était enfouie dans le code de l’interface graphique, ce qui la rendait inaccessible depuis un script.

2.3.2 Refactorisation et API

J’ai extrait l’intégralité de la logique algorithmique dans un module Python indépendant : `mesh_boolean_api.py`. Ce module expose une interface claire permettant d’effectuer n’importe quelle opération booléenne sans ouvrir SALOME en mode graphique :

```
from meshbooleanapi import mesh_boolean_api
result = mesh_boolean_api.Operation(mesh_A, mesh_B, algo=mesh_boolean_api.ALGO)
```

L’interface graphique PyQt est devenue un simple *wrapper* appelant cette API, sans contenir de logique métier. Cette séparation des responsabilités améliore la maintenabilité et rend le plugin testable indépendamment de l’interface.

2.3.3 Gestion des ressources et qualité de code

Chaque opération booléenne génère plusieurs fichiers intermédiaires (formats `.obj`, `.stl`, `.off`) stockés dans le répertoire temporaire système. Bien qu’utiles dans certains cas pour déboguer

un mauvais fonctionnement des algorithmes, ces fichiers sont sans valeur pour l'utilisateur final et leur accumulation peut rapidement saturer l'espace disque.

J'ai résolu ce problème en deux étapes. D'abord, **tous les fichiers intermédiaires sont désormais regroupés dans un répertoire temporaire dédié**, créé automatiquement au début de chaque calcul. Ensuite, ce répertoire est encapsulé dans un **context manager** Python (`with / try`), qui garantit la suppression dans tous les cas, y compris lors d'une exception ou d'une annulation par l'utilisateur.

J'ai également appliqué **Pylint**⁶ sur l'ensemble du code pour corriger les violations de style, améliorer la lisibilité et uniformiser les conventions de nommage. Des habitudes ont été données par mes tuteurs, de toujours bien nettoyer mon code, bien le commenter et ajouter des loggers pour des potentiels futurs debugs. Enfin, j'ai supprimé un élément graphique superflu de la boîte de dialogue principale qui alourdissait l'interface sans apporter de valeur fonctionnelle.

2.3.4 Mode verbose et facilitation du débogage

Une contribution que j'ai jugée importante pour la maintenabilité à long terme du plugin est l'ajout d'un **mode verbose conditionnel**. Lorsque la variable d'environnement `SALOME_VERBOSE=1` ou `DEBUG_PLUGIN=1` est définie, le plugin affiche des informations détaillées sur le déroulement du calcul : appels aux algorithmes, chemins des fichiers intermédiaires, codes de retour des processus. En conditions normales, aucun message superflu n'est affiché. Ce mécanisme, inspiré des pratiques standard du développement SALOME, facilite considérablement le diagnostic des erreurs sans polluer l'interface en production.

2.4 Intégration au Dump Study

2.4.1 Fonctionnement du Dump Study dans SALOME

Le *dump study* est une fonctionnalité native de SALOME qui enregistre automatiquement l'ensemble des actions effectuées par l'utilisateur dans l'interface graphique sous la forme d'un script Python rejouable. Cela permet de reproduire une session de travail complète sans interaction manuelle, ce qui est essentiel pour l'automatisation et la traçabilité des simulations. À titre d'illustration, la préparation d'un maillage complexe peut nécessiter plusieurs heures de manipulations interactives. Grâce au *dump study*, l'ingénieur peut sauvegarder l'intégralité de ce processus de construction. Si un collaborateur souhaite ensuite appliquer ce même maillage à une géométrie de dimensions différentes, il lui suffit d'éditer les coordonnées paramétriques directement dans le script Python. L'ensemble des opérations peut alors être relancé de manière entièrement automatisée, évitant ainsi de reproduire les étapes manuelles.

2.4.2 Problème identifié

Sans intégration spécifique, les exports de fichiers intermédiaires réalisés par le plugin étaient capturés tels quels par le système d'enregistrement de SALOME. Le script généré contenait alors des appels à des chemins de fichiers temporaires qui n'existaient plus lors de la relecture, et des lignes de codes inutiles rendant le script **inutilisable hors de la session originale**.

2.4.3 Solution implémentée

J'ai mis en place deux mécanismes complémentaires. D'abord, la **suspension de l'enregistrement** pendant les étapes d'export intermédiaires, en utilisant les commandes de contrôle du dump study de SALOME. Ensuite l'**injection directe d'appels API propres** dans le script généré, de sorte que chaque opération booléenne réalisée via l'interface graphique apparaisse dans le dump comme un appel Python lisible et autonome :

```

from meshbooleanplugin import mesh_boolean_api
operation_1 = mesh_boolean_api.Operation(
    Mesh_1.GetMesh(),
    Mesh_2.GetMesh(),
    algo=mesh_boolean_api.ALGO,
    name="my_operation"
)

```

Deux détails d'implémentation méritent d'être soulignés. Premièrement, chaque fonction de l'API accepte un paramètre optionnel `name` permettant à l'utilisateur de nommer directement le maillage de sortie depuis le script Python, sans interaction avec l'interface graphique.

Deuxièmement, la gestion des objets SALOME est assurée par un test `hasattr` sur chaque maillage d'entrée : si l'objet possède une méthode `.GetMesh()`, celle-ci est appelée automatiquement pour en extraire le maillage compatible avec l'API. Cela permet à l'API d'accepter indifféremment un objet de l'arbre d'étude SALOME - tel que sélectionné dans l'interface graphique - ou un objet maillage Python pur, garantissant une utilisation cohérente dans les deux contextes.

Le dump study est ainsi devenu un pont opérationnel entre l'interface graphique et l'API Python : toute session de travail peut être exportée, modifiée et relancée de manière entièrement automatisée.

La figure 2.3 montre un extrait d'un script généré par le dump study après une session utilisant le plugin.

```

from meshbooleanplugin import mesh_boolean_api
union_1 = mesh_boolean_api.Union(Mesh_1.GetMesh(), Mesh_tetra_coarse.GetMesh(), algo = mesh_boolean_api.CGAL)
intersection_1 = mesh_boolean_api.Intersection(Mesh_1.GetMesh(), Mesh_tetra_coarse.GetMesh(), algo = mesh_boolean_api.VTK)
difference_1 = mesh_boolean_api.Difference(intersection_1, Mesh_tetra_coarse.GetMesh(), algo = mesh_boolean_api.CORK)

```

FIGURE 2.3 – Extrait d'un script Python généré par le dump study de SALOME après une session incluant trois opérations booléennes. Les appels à l'API sont propres et directement rejouables sans interaction manuelle.

2.5 Tests et validation

La validité des algorithmes a été vérifiée par un ensemble de **tests automatisés** implémentés avec le module `unittest` de Python, en utilisant le mécanisme de `subTest` pour couvrir les combinaisons entre les six algorithmes et les trois opérations (union, intersection, différence), soit **18 sous-tests** au total dans le cas où tous les algorithmes sont disponibles.

Une conception importante du script de test est qu'il **détecte dynamiquement les algorithmes disponibles** en vérifiant la présence de leurs variables d'environnement respectives au moment de l'exécution. Ainsi, seuls les algorithmes effectivement installés et accessibles sont testés, ce qui rend la suite de tests portable d'un système d'exploitation à l'autre. Dans mon environnement de développement, 5 algorithmes sur 6 sont disponibles, ce qui conduit à l'exécution de 15 sous-tests au total. La précision des résultats a été validée avec une marge d'erreur de 5×10^{-4} sur l'aire calculée. Ces tests peuvent être exécutés directement en ligne de commande après initialisation du **SALOME context**, sans ouvrir l'interface graphique.

La figure 2.4 illustre l'exécution de la suite de tests en ligne de commande.

```

Total sub-tests ran : 15
.
-----
Ran 3 tests in 13.180s

OK

```

FIGURE 2.4 – Résultat de l’exécution des 15 sous-tests automatisés du MeshBooleanPlugin en ligne de commande, couvrant les 5 algorithmes disponibles et les trois opérations booléennes.

2.6 Cas d’usage scientifiques et perspectives

Les opérations booléennes sur maillage répondent à un besoin concret dans les workflows de simulation numérique au CEA, en particulier lorsqu’aucun modèle CAO n’est disponible et que le maillage constitue la seule représentation géométrique exploitable - comme c’est le cas avec des données issues de la tomographie. Dans ce contexte, le **MeshBooleanPlugin** peut représenter une alternative open source à des outils commerciaux comme **MG-Cleaner** [17], actuellement utilisés au CEA pour des opérations similaires de manipulation de maillages.

Deux cas d’usage concrets, actuellement traités avec MG-Cleaner, illustrent le potentiel d’intégration du plugin dans des workflows réels.

Le premier concerne la **modélisation de batteries lithium-ion**. Une cellule de batterie est constituée de plusieurs composants de géométries imbriquées - électrodes, séparateur, collecteurs de courant. Disposer d’un outil capable d’effectuer des opérations booléennes directement sur les maillages de ces composants permettrait de les assembler, de les identifier individuellement par groupe, et de produire un maillage global cohérent utilisable dans des simulations thermiques ou électrochimiques. La figure 2.5 illustre un exemple de ce type de microstructure poreuse d’électrode utilisée dans ce contexte de recherche.

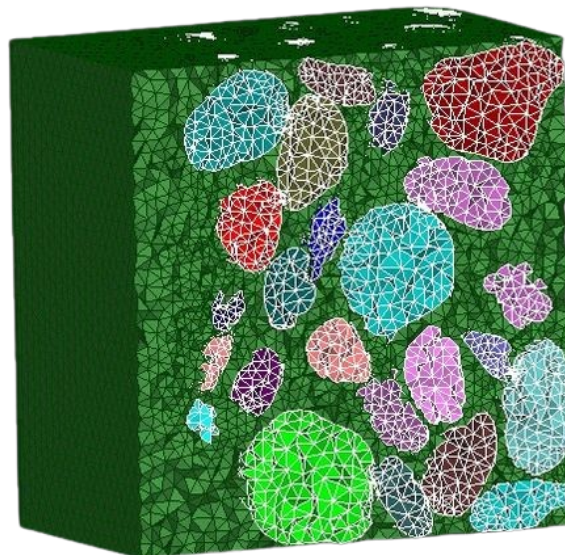


FIGURE 2.5 – Exemple de microstructure poreuse d’électrode de batterie lithium-ion. Ce type de géométrie complexe est un cas d’usage cible pour les opérations booléennes entre maillages dans le cadre des recherches sur le stockage d’énergie au CEA.

Le second cas porte sur la **modélisation du béton et de ses agrégats**. Il s'agit de modéliser le comportement mécanique d'un matériau composite hétérogène constitué d'une matrice de béton et d'inclusions solides de formes irrégulières. L'opération d'intersection entre un maillage volumique représentant la matrice et les maillages des agrégats permettrait de produire directement un maillage exploitable pour des simulations de résistance mécanique, sans passer par un outil commercial. La figure 2.6 illustre ce type de configuration géométrique.

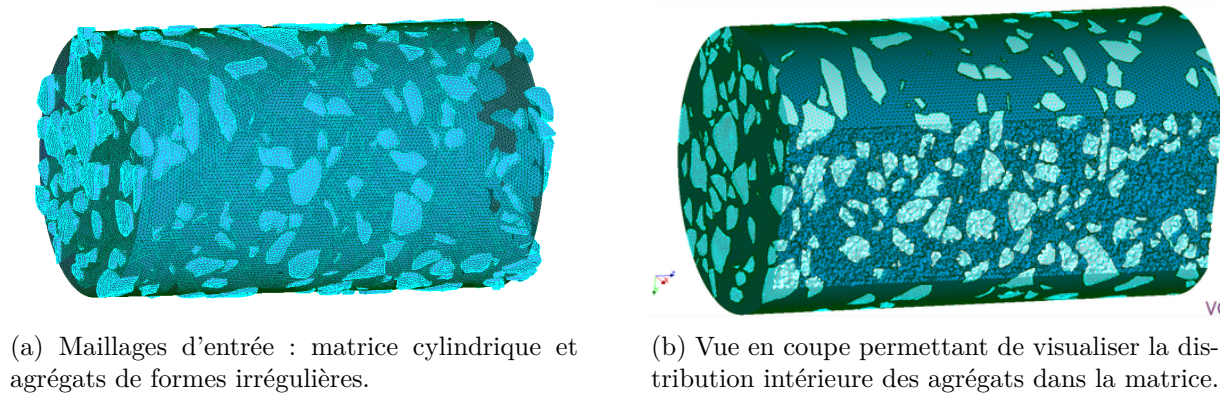


FIGURE 2.6 – Modélisation du béton et de ses agrégats : configuration géométrique cible pour une application des opérations booléennes entre maillages dans le cadre de simulations de résistance mécanique de matériaux composites. Images générées avec SALOME.

Ces deux cas d'usage, identifiés avec mes encadrants, sont actuellement traités au CEA avec **MG-Cleaner**⁷. Ils n'ont pas encore été testés avec le MeshBooleanPlugin dans le cadre de cette première année, mais constituent des perspectives concrètes de validation et d'intégration pour les années suivantes. La stabilité des algorithmes sur ces géométries complexes devra notamment être évaluée, certains algorithmes étant plus adaptés que d'autres selon la topologie des maillages en entrée.

3 | Deuxième trimestre : PolyMeshPlugin

3.1 Présentation du plugin et état initial

Le **PolyMeshPlugin** est le second plugin sur lequel j'ai travaillé au cours de cette première année. Son rôle est de transformer n'importe quel maillage volumique ou surfacique fermé en un **maillage polyédrique**, un type de maillage particulièrement adapté aux simulations en mécanique des fluides numériques (CFD⁸). Le plugin intègre trois algorithmes C++ distincts, chacun produisant un type de maillage polyédrique différent :

- o **Polydual** : construit le maillage polyédrique par dualisation du maillage d'entrée. Cet algorithme s'appuie sur les utilitaires de la bibliothèque **OpenFOAM**.
- o **Cfmesh** : générateur de maillage polyédrique basé, comme Polydual, sur la bibliothèque **OpenFOAM**, permettant notamment de spécifier une taille maximale de cellule.
- o **Geogram** : utilise des diagrammes de Voronoï, a une option pour générer un maillage à partir de points seeds, avec la possibilité de contrôler finement le niveau de raffinement.

La figure 3.1 illustre les trois types de maillages polyédriques produits par chacun des algorithmes sur une même géométrie d'entrée.

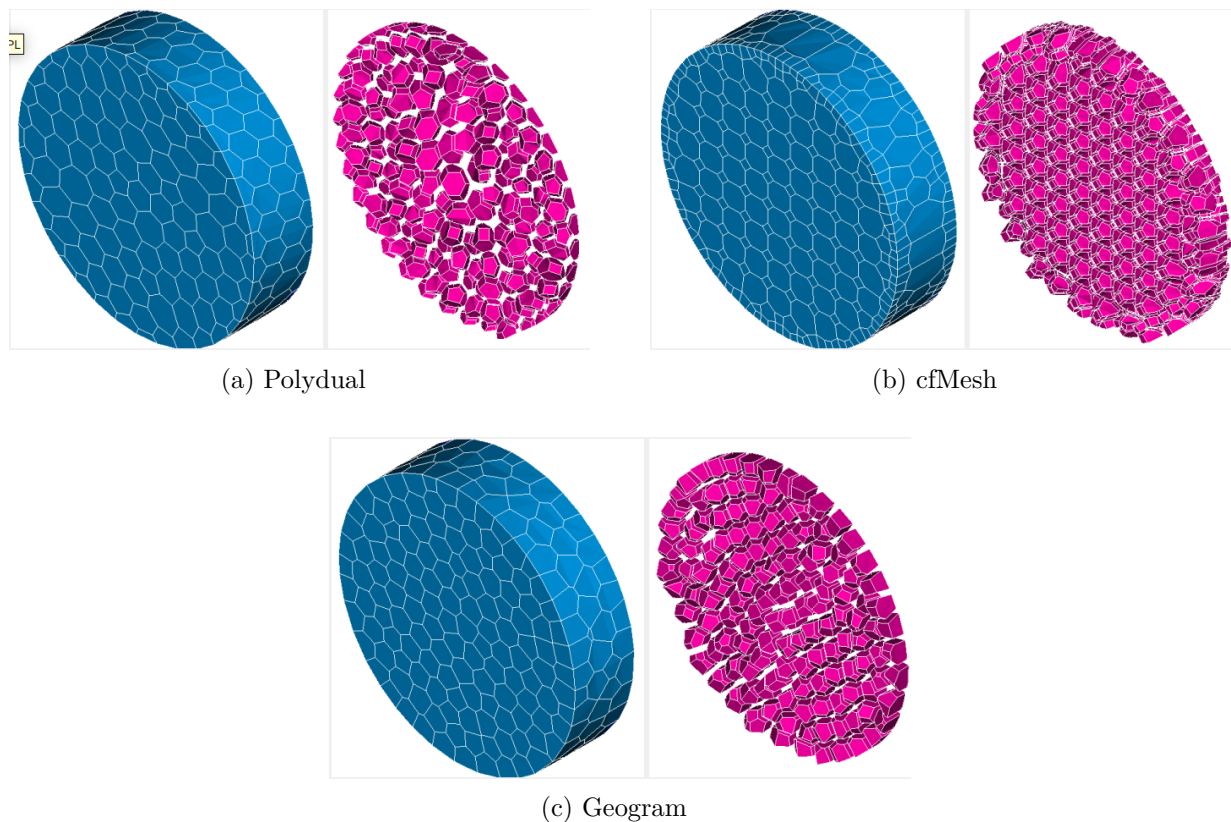


FIGURE 3.1 – Comparaison des trois types de maillages polyédriques générés par le PolyMeshPlugin sur une même géométrie d'entrée. Chaque algorithme produit une topologie cellulaire différente. Le rendu bleu correspond au maillage surfacique, tandis que le rendu rose présente le maillage volumique en vue éclatée et en coupe.

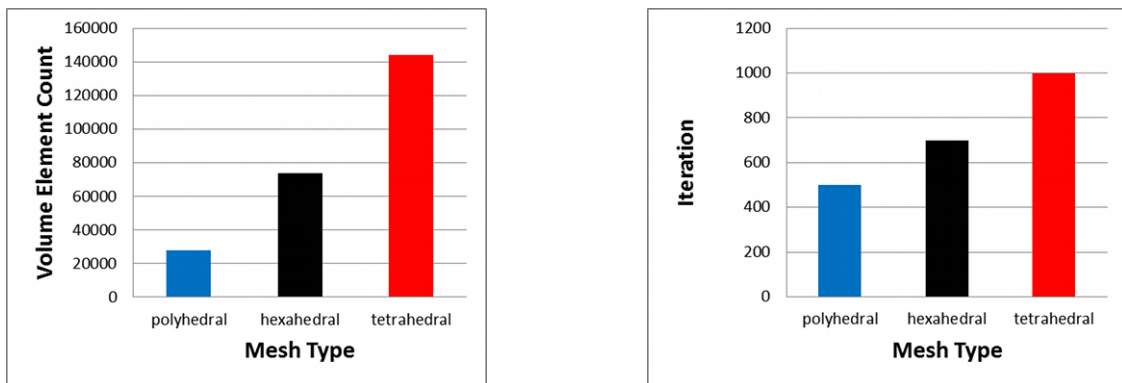
3.2 Intérêt du maillage polyédrique

Avant d’entrer dans les détails techniques, il convient de justifier l’intérêt scientifique du maillage polyédrique par rapport aux types de maillages plus classiques.

En simulation numérique, trois grands types de maillages volumiques sont couramment utilisés : tétraédrique, hexaédrique et polyédrique. Le maillage **tétraédrique** est le plus facile à générer automatiquement sur des géométries complexes mais, suivant les modèles physiques employés, il peut produire les résultats CFD les moins précis pour un nombre de cellules donné. Le maillage **hexaédrique** offre de meilleures performances en CFD mais est plus difficile à générer sur des géométries non triviales. Le maillage **polyédrique** représente un compromis intéressant entre ces deux options.

Concrètement, une étude comparative publiée par Symscape [11] sur un cas simple de conduite (*duct*) montre que le maillage polyédrique contient environ **cinq fois moins de cellules** que le maillage tétraédrique équivalent, tout en atteignant le même niveau de convergence pour une simulation CFD en **deux fois moins d’itérations**. Cette réduction de nombre de cellules se traduit directement par une diminution des temps de calcul, ce qui est un enjeu majeur pour les simulations industrielles réalisées au CEA.

La figure 3.2 illustre ces avantages quantitatifs tirés de cette étude.



(a) Nombre d’éléments volumiques selon le type de maillage.

(b) Nombre d’itérations pour atteindre la convergence (10^{-4}) selon le type de maillage.

FIGURE 3.2 – Comparaison quantitative entre maillages tétraédrique, hexaédrique et polyédrique sur un cas de conduite simple. Source : [11].

Du point de vue numérique, les cellules polyédriques présentent une meilleure **orthogonalité** que les tétraèdres, ce qui réduit les erreurs de troncature lors du calcul des gradients et améliore la précision globale de la simulation. Elles s’adaptent également mieux aux géométries complexes que les hexaèdres, qui nécessitent un travail de découpage par blocs souvent manuel.

3.3 État du plugin à la reprise

Le PolyMeshPlugin avait été initié par un stagiaire de cinq mois avant que je le reprenne en février 2026. Le prototype fonctionnel existait mais présentait plusieurs limitations bloquantes :

- o La méthode utilisée pour communiquer avec les exécutables des algorithmes C++, basée sur les bibliothèques **pexpect** (Linux) et **wexpect** (Windows), nécessitait un script d’installation post-déploiement sur Windows et présentait des comportements instables selon les environnements.

- o L'interface graphique se **figeait** après la fin du calcul, au moment du chargement du maillage dans SALOME, sans aucun retour visuel pour l'utilisateur.
- o Il n'existait aucune gestion des erreurs : un calcul produisant un maillage vide ou incomplet échouait silencieusement, sans message explicatif.
- o Le pipeline de conversion pour l'algorithme Geogram passait par un format intermédiaire `.ovm` parsé par une fonction Python de centaines de lignes, peu robuste et difficile à maintenir.

Mes travaux sur ce plugin se sont étalés de février à avril 2026.

3.4 Refactorisation multiplateforme

3.4.1 Remplacement de `pexpect` par `subprocess`

La première action a consisté à remplacer la méthode `pexpect/wexpect` par le module **`subprocess`** de la bibliothèque standard Python - la même approche que celle adoptée avec succès sur le `MeshBooleanPlugin`. Ce remplacement présente trois avantages : aucune dépendance externe à installer, un comportement identique sur Linux et Windows, et une interface de programmation plus simple et mieux documentée.

3.4.2 Barre de progression en temps réel

Le remplacement de `pexpect` introduisait cependant une difficulté : la bibliothèque originale permettait d'ouvrir un terminal fictif et lire la sortie en temps réel, ce qui était utilisé pour mettre à jour une barre de progression. Avec `subprocess` en mode standard, reproduire cela nécessitait une implémentation plus complexe.

J'ai résolu ce problème par une **lecture caractère par caractère** de la sortie standard du processus en terminal, dans un thread séparé pour ne pas bloquer l'interface. Les caractères sont accumulés dans un buffer, vidé à chaque espace ou retour à la ligne. Chaque ligne est ensuite analysée pour extraire un pourcentage d'avancement, et chaque étape franchie par l'algorithme déclenche une mise à jour de la barre de progression affichée dans l'interface PyQt. L'utilisateur dispose ainsi d'un retour visuel continu pendant toute la durée du calcul.

La figure 3.3 illustre l'interface du plugin pendant un calcul en cours.

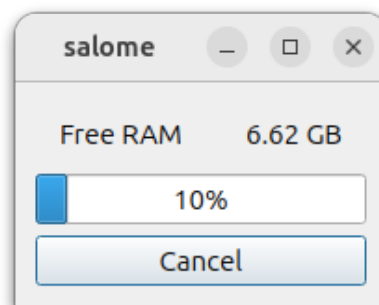


FIGURE 3.3 – Boîte de progression affichée pendant le calcul du PolyMeshPlugin, mise à jour en temps réel par lecture caractère par caractère de la sortie du processus.

3.4.3 Gestion des erreurs et robustesse

Trois mécanismes de gestion des erreurs ont été ajoutés pour améliorer la fiabilité du plugin en conditions réelles.

Détection du maillage vide pour Cfmesh : l'algorithme Cfmesh peut produire un maillage vide si le paramètre de taille maximale de cellule est mal configuré. J'ai ajouté une vérification explicite du nombre de points générés à l'issue du calcul. Si ce nombre est nul, une boîte de dialogue d'avertissement est affichée avec un message clair : *"The generated mesh is empty (0 points). Please adjust 'Max cell size' and try again."* Cela évite à l'utilisateur de chercher pourquoi SALOME n'affiche rien après le calcul apparemment réussi.

La figure 3.4 illustre ce message d'avertissement.

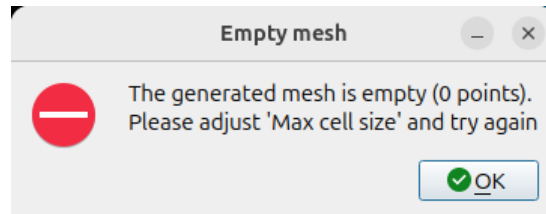


FIGURE 3.4 – Boîte de dialogue d'avertissement affichée par le PolyMeshPlugin lorsque cfMesh génère un maillage vide. Le message guide l'utilisateur vers le paramètre à ajuster.

Avertissement sur les transferts de groupes partiels : lors de la conversion d'un maillage en maillage polyédrique, les groupes définis sur le maillage d'entrée sont transférés au maillage de sortie. Si ce transfert est incomplet, le plugin affiche désormais un avertissement explicite au lieu d'échouer silencieusement.

Boîte de chargement post-calcul : une fois le calcul terminé, le fichier `.med` résultant doit être importé et affiché dans l'object browser de SALOME, ce qui peut prendre quelques secondes. J'ai ajouté une boîte de dialogue de chargement pendant cette phase, afin que l'utilisateur sache que l'opération n'est pas terminée et n'interagisse pas avec l'interface pendant l'import, évitant ainsi tout potentiel crash.

En parallèle, mes encadrants et moi avons optimisé le lancement du module Paraview en ligne de commande, évitant l'import de `paraview.simple` au démarrage de SMESH qui causait un ralentissement de plusieurs secondes de l'interface.

3.5 Refonte architecturale du pipeline Geogram

3.5.1 Problème initial

Le pipeline de conversion pour l'algorithme Geogram était le point le plus critique du plugin sur le plan de la maintenabilité. Dans l'état initial, le flux de données suivait la chaîne suivante : Geogram produisait un fichier au format `.ovm`, qui était ensuite parsé par du code Python avant d'être importé dans SALOME.

Le format `.ovm` (OpenVolumeMesh) est un format de fichier ASCII⁹ [16] décrivant des maillages volumiques généraux. Son parsing en Python représentait quelques centaines de lignes de code lentes, difficile à déboguer et sensible aux variations de format produites par différentes versions de Geogram.

La figure 3.5 compare l'ancien et le nouveau pipeline de conversion.

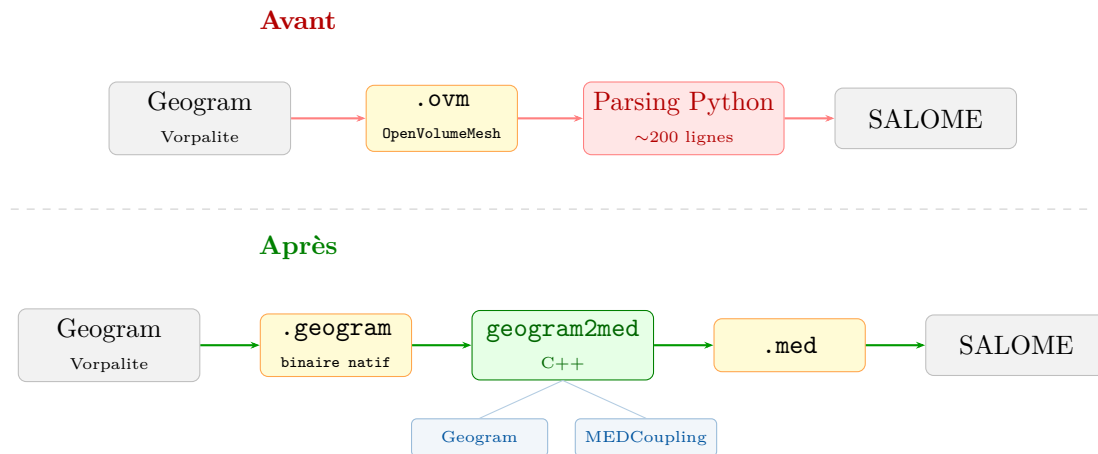


FIGURE 3.5 – Refonte du pipeline de conversion Geogram : suppression du format intermédiaire `.ovm` et du parsing Python fragile au profit d’une conversion C++ directe vers `.med`, s’appuyant sur les bibliothèques Geogram et MEDCoupling. Les boîtes jaunes représentent les fichiers, les boîtes colorées les algorithmes de conversion.

3.5.2 Conception et développement de `geogram2med`

Pour résoudre ce problème, j’ai conçu et développé entièrement un **exécutable C++ dédié**, nommé `geogram2med`, dont la compilation est gérée par un fichier `CMakeLists.txt` que j’ai également écrit. Cet exécutable s’appuie sur deux bibliothèques :

- o **Geogram** : développée par l’Inria¹⁰ [15], cette bibliothèque permet de lire nativement le fichier `.geogram` binaire produit par l’algorithme Vorpalite.
- o **MEDCoupling** : la bibliothèque C++ de SALOME pour la manipulation et l’export de maillages au MED¹¹ (`.med`), nativement supporté par la plateforme.

Fonctionnement interne de `geogram2med`

L’exécutable suit une séquence de traitement précise, directement ancrée dans la structure du format `.geogram`. Geogram¹² stocke les informations de connectivité polyédrique sous la forme de **facettes** 2D auxquelles est associé un attribut entier `cell_id`, indiquant à quelle cellule polyédrique appartient chaque facette. C’est cet attribut qui est produit lorsque l’option `poly:generate_ids=true` est activée dans Vorpalite.

Le traitement s’effectue en plusieurs étapes :

1. **Chargement** du fichier `.geogram` via la bibliothèque Geogram et extraction des coordonnées de tous les nœuds dans un tableau `DataArrayDouble` de MEDCoupling.
2. **Construction du dictionnaire** : parcours de toutes les facettes 2D et regroupement par `cell_id` dans une structure `std::map<int, vector<index_t>`.
3. **Reconstruction des polyèdres** : pour chaque cellule du dictionnaire, concaténation des indices de nœuds de ses facettes en les séparant par des `-1`, conformément à la convention MEDCoupling pour la norme `NORM_POLYHED`.
4. **Calcul de la surface** (*skin*) du volume via `computeSkin()`, qui produit le maillage 2D de frontière nécessaire à SALOME pour l’affichage et la définition de conditions aux limites.
5. **Export** du fichier `.med` contenant le maillage volumique 3D, le maillage surfacique 2D et les groupes associés.

3.5.3 Résultats et bénéfices

Cette refonte a permis de **supprimer environ 200 lignes de Python fragile**, de rendre le pipeline plus rapide grâce à la lecture binaire native du format `.geogram`, et d'améliorer significativement la maintenabilité globale du code source du plugin. Sur un maillage de 500 cellules polyédriques, le temps total de traitement est passé de 1.24s à 1.16s, un gain amené à croître significativement sur des géométries industrielles de plus grande taille.

3.6 Cas test et validation

La validation du plugin a été effectuée sur plusieurs géométries complexes représentatives des cas d'usage réels au CEA. L'option d'export de seeds de Geogram permet notamment de conserver le même niveau de raffinement dans le maillage polyédrique que dans le maillage d'entrée. Le cas le plus démonstratif est celui d'une **pièce mécanique**, une géométrie caractérisée par de fortes variations de courbures. Ce type de composant est couramment utilisé pour valider la robustesse des algorithmes de maillage face à des topologies non triviales.

Pour l'algorithme Geogram avec export de seeds, le maillage polyédrique obtenu présente une distribution de cellules directement corrélée au raffinement local du maillage tétraédrique d'entrée : les zones initialement plus raffinées conservent une densité de cellules plus élevée dans le maillage polyédrique.

Afin de mettre en évidence cette propriété, la figure 3.6 présente à droite une **superposition des deux maillages**. Cette visualisation permet d'observer clairement la dualité géométrique : les nœuds du maillage tétraédrique initial constituent exactement les centres des nouvelles cellules polyédriques. Ce comportement est particulièrement utile pour les simulations où la conservation du raffinement local est critique, par exemple pour capter précisément les contraintes physiques autour des perçages. **Ces résultats confirment la maturité du plugin pour une utilisation en production.** L'ensemble des développements a été validé par *pull requests* sur le dépôt interne du PolyMeshPlugin et sera intégré à la prochaine version publique de SALOME après passage des tests d'intégration.



FIGURE 3.6 – Validation du PolyMeshPlugin sur une pièce mécanique. La vue superposée (à droite) démontre que l'algorithme Geogram avec export de seeds conserve parfaitement le raffinement local : chaque point seed du maillage tétraédrique devient le centre d'une cellule polyédrique.

4 | Responsabilité sociétale et transition écologique au CEA

4.1 Contexte et enjeux DDRS

Pour un organisme public de recherche comme le CEA, dont les travaux touchent directement aux questions énergétiques et climatiques, la DDRS¹³ n'est pas périphérique : elle est constitutive de la mission même de l'institution.

En France, le numérique représente aujourd'hui environ **4,4 % des émissions nationales de gaz à effet de serre** [13], un chiffre en progression constante et qui pourrait tripler d'ici 2050 sans actions correctives. Dans ce contexte, les organismes publics de recherche sont soumis à une double responsabilité : maîtriser l'empreinte de leur propre activité numérique, et mettre leur expertise au service de la transition écologique.

4.2 La mission énergétique du CEA comme réponse aux enjeux climatiques

Le CEA a été historiquement fondé autour de la question de l'énergie. Pour atteindre les objectifs de neutralité carbone à l'horizon 2050, le CEA travaille sur de nombreuses briques du système de production d'énergies bas carbone : soutien au parc nucléaire actuel et développement de petits réacteurs modulaires (SMR), recherche sur le solaire photovoltaïque, l'hydrogène et le stockage de l'énergie.

Cette orientation est particulièrement visible au sein de la Direction des Énergies (DES), à laquelle est rattaché mon laboratoire d'accueil. L'Institut des Sciences Appliquées et de la Simulation pour les énergies bas carbone (ISAS), dont dépend le Service de Génie Logiciel (SGLS) et le laboratoire LESIM, a précisément pour vocation de développer les outils numériques de simulation au service de cette transition énergétique. La simulation numérique y joue un rôle central : elle permet de concevoir et d'optimiser des réacteurs, des batteries ou des systèmes de stockage sans recourir à des expériences physiques coûteuses en matériaux et en énergie.

4.3 L'open source comme pratique de numérique responsable

Une dimension importante de la politique DDRS du CEA dans le domaine logiciel est l'engagement en faveur de l'**open source**. SALOME est publié en open source depuis 2016, ce qui présente plusieurs avantages du point de vue de la responsabilité sociétale :

- o **Mutualisation des développements** : les contributions de différents acteurs (CEA, EDF, OpenCascade, communauté académique) évitent la duplication d'efforts et réduisent le coût global de développement.
- o **Transparence et auditabilité** : le code source étant accessible, les algorithmes utilisés dans les simulations peuvent être vérifiés et validés par la communauté scientifique internationale.
- o **Réduction de la dépendance aux logiciels commerciaux** : SALOME offre une alternative open source à des outils propriétaires, réduisant ainsi la dépendance technologique et les coûts de licence pour les institutions académiques et industrielles.
- o **Accessibilité** : en rendant la plateforme gratuite et ouverte, le CEA contribue à démocratiser l'accès aux outils de simulation numérique, y compris pour des pays ou des

institutions disposant de ressources limitées.

Mes deux contributions suivront ce modèle : le MeshBooleanPlugin rejoindra la prochaine version publique, tandis que le PolyMeshPlugin, actuellement en interne (CEA/EDF) pour test et retour d'expérience, sera ouvert à la communauté lors de la version suivante.

4.4 Regard critique sur les pratiques observées

4.4.1 Points forts

La vocation même de mon laboratoire — développer des outils de simulation pour les énergies bas carbone — constitue une contribution structurelle aux enjeux de transition écologique. Le CEA n'intègre pas la durabilité comme un ajout externe à ses activités, c'est le cœur de la mission de la Direction des Énergies.

L'engagement open source de SALOME est un exemple concret de numérique responsable. Partager le code, accepter des contributions extérieures et maintenir une documentation publique reflète une posture d'ouverture qui va au-delà du simple affichage.

4.4.2 Points d'amélioration et limites observées

Cependant, quelques tensions méritent d'être signalées. Les simulations numériques elles-mêmes, bien qu'elles réduisent le recours aux expériences physiques, consomment des ressources informatiques significatives. Les calculs HPC¹⁴ (High Performance Computing) réalisés au CEA mobilisent des clusters de calcul dont la consommation énergétique est considérable. Au niveau des laboratoires, la mesure de l'**empreinte carbone du calcul scientifique** reste peu systématisée et peu intégrée aux pratiques quotidiennes de développement, bien que des initiatives aient été menées avec plusieurs projets, ce qui montre quand même qu'elle progresse.

La **politique de support multi-plateformes de SALOME** constitue un autre axe de réflexion. Actuellement, la plateforme est compilée et maintenue pour un grand nombre de systèmes d'exploitation ainsi qu'un binaire universel Linux. Cette multiplicité implique des centaines d'heures de compilation, de test et de maintenance par version publiée, mobilisant des ressources humaines et informatiques considérables. Une évolution vers un **binaire universel unique** compatible avec l'ensemble des distributions Linux représenterait un gain significatif en termes de sobriété numérique et de maintenabilité, sans dégrader l'accessibilité de la plateforme pour les utilisateurs.

4.5 Lien entre mes missions et les enjeux DDRS

En conclusion de cette section, je souhaite souligner le lien direct entre mes travaux techniques et les enjeux de développement durable.

Le développement du PolyMeshPlugin, et notamment l'amélioration de la qualité des maillages polyédriques, contribue à des simulations CFD plus précises et plus efficaces. Des maillages de meilleure qualité convergent plus rapidement, ce qui réduit le temps de calcul et donc la consommation énergétique associée. C'est un exemple de ce que le CEA-List appelle le *numérique au service du développement durable* : utiliser les technologies numériques pour optimiser d'autres processus et réduire leur impact environnemental global.

De même, la refactorisation du MeshBooleanPlugin — en extrayant une API réutilisable et en intégrant le dump study — améliore la **réutilisabilité du code**, réduisant ainsi le besoin de redévelopper des fonctionnalités existantes et allongeant la durée de vie utile du logiciel. C'est une forme de sobriété logicielle qui s'inscrit dans les principes du numérique responsable.

5 | Bilan et retour d'expérience

5.1 Compétences techniques acquises

Cette première année d'apprentissage m'a permis de développer un ensemble de compétences techniques directement applicables dans un contexte industriel et de recherche.

5.1.1 Python et architecture logicielle

Mes contributions au `MeshBooleanPlugin` m'ont permis de maîtriser plusieurs aspects avancés de Python : la gestion de processus avec `subprocess`, la programmation concurrente avec `QThread`, les context managers pour la gestion des ressources, et la conception d'une API claire et documentée. J'ai également pris l'habitude systématique du nettoyage de code avec `Pylint` et de l'écriture de tests automatisés avec `unittest` — des pratiques que j'avais abordées en cours mais que je n'avais jamais appliquées de manière rigoureuse sur un projet réel.

5.1.2 C++ et MEDCoupling

Le développement de `geogram2med` représente ma première contribution significative en C++ dans un contexte professionnel. J'ai dû apprendre à utiliser la bibliothèque `MEDCoupling`, en lisant le code source et en m'appuyant sur les conseils de mes encadrants. Cette expérience m'a appris à être autonome face à une bibliothèque nouvelle, à lire une documentation et à faire des choix d'implémentation justifiés. J'ai également suivi une **formation C++ scientifique moderne de 21 heures dispensée par le CNRS**, qui m'a donné des bases solides sur les bonnes pratiques du langage dans un contexte de calcul scientifique.

5.1.3 Git et workflow collaboratif

Je connaissais Git très brièvement avant d'arriver, uniquement pour un usage personnel et superficiel. Depuis mon premier jour au CEA, je l'utilise quotidiennement - pour chaque contribution aux plugins, pour documenter mon travail, pour gérer mes branches de développement. Travailler sur un dépôt partagé avec des équipes du CEA et d'EDF m'a confronté à des situations que je n'avais jamais rencontrées : conflits de merge, rebase interactif, pull requests revues par des développeurs seniors. La résolution du conflit avec les imports `salome.kernel` lors de ma première pull request a été un moment formateur - j'ai passé plusieurs heures avec mes encadrants à comprendre le problème et à utiliser le rebase interactif pour le résoudre proprement.

Aujourd'hui, Git fait partie intégrante de ma façon de travailler, y compris à l'école où je l'utilise désormais pour tous mes projets académiques. C'est l'une des compétences transférables les plus concrètes que cette année m'a apportées.

5.1.4 Formation et montée en compétence sur SALOME

Dès mes premières semaines au CEA, j'ai suivi une formation SALOME en ligne afin de me familiariser rapidement avec la plateforme avant de contribuer à son code. Cette formation couvre les fonctionnalités principales de la plateforme : navigation dans l'interface, manipulation de géométries dans le module GEOM, génération de maillages dans SMESH et post-traitement dans Paravis. Cette étape préalable m'a permis d'acquérir une vision d'ensemble de l'écosystème SALOME et de comprendre le contexte dans lequel s'inscrivent mes développements.

5.1.5 Environnement Linux

Je n'avais absolument aucune expérience de Linux avant d'arriver au CEA. J'ai passé les premières semaines à naviguer dans un terminal, essayer de comprendre et configurer l'environnement de développement. J'ai appris progressivement, commande par commande, en m'appuyant sur mes encadrants et sur la pratique quotidienne.

Aujourd'hui, je suis complètement autonome sur Linux. Je maîtrise l'environnement en ligne de commande, la gestion des processus, la configuration des variables d'environnement et les outils de développement associés. Cette progression a été suffisamment marquante pour que je décide de passer mon ordinateur personnel sous Ubuntu - une décision qui illustre mieux que tout discours le niveau d'aisance que j'ai acquis cette année.

5.1.6 Veille technologique et autonomie d'apprentissage

Au-delà des formations structurées, cette année m'a appris à apprendre de manière autonome. Face à une bibliothèque inconnue comme MEDCoupling, ou à un comportement inattendu de `multiprocessing` sur Windows, j'ai développé le réflexe de consulter le code source directement plutôt que d'attendre une documentation exhaustive. La lecture de code existant — celui de SALOME, des plugins, des bibliothèques tierces — est devenue l'une de mes premières sources d'apprentissage, une habitude que je n'avais pas du tout avant cette année.

5.2 Compétences transversales et humaines

5.2.1 Autonomie et organisation

Le rythme de l'alternance demande une capacité à reprendre rapidement le fil d'un travail après une interruption. J'ai appris à tenir une documentation de mon travail au quotidien, à noter les décisions techniques prises et les raisons qui les ont motivées, pour pouvoir reprendre efficacement après chaque période scolaire.

J'ai également appris à mieux structurer mon travail face à un problème complexe. Au début de l'année j'avais tendance à vouloir tout résoudre d'un coup. Désormais, j'essaie de découper systématiquement une nouvelle mission en sous-tâches réalisables. Cette méthode m'évite de rester bloqué trop longtemps, me permet de faire des validations fréquentes (commits) et de montrer des avancées régulières lors des points avec mes tuteurs.

Sur le plan des outils de communication scientifique, cette année m'a également conduit à maîtriser le $\text{\LaTeX}$ pour la rédaction de ce rapport. Apprendre à structurer un document long, gérer les références croisées, produire des schéma TikZ et maintenir une mise en page cohérente sur plusieurs dizaines de pages constitue une compétence à part entière, directement utile pour la rédaction de rapports techniques et publications scientifiques.

5.2.2 Communication technique

Travailler en équipe avec des ingénieurs seniors m'a appris à formuler mes questions de manière précise. Au début, j'avais tendance à poser des questions trop larges ou mal cadrées. Progressivement, j'ai appris à préparer mes questions : d'abord chercher par moi-même, documenter ce que j'avais essayé, identifier précisément ce qui bloquait, puis exposer le problème clairement. Cette habitude a considérablement amélioré la qualité de mes échanges avec mes encadrants.

5.2.3 Intégration dans un environnement professionnel

Arriver au CEA, une structure historique et imposante, était assez impressionnant au départ. J'ai dû m'adapter rapidement aux codes d'un grand laboratoire : participer activement aux

réunions d'équipe, faire des points d'avancement réguliers et comprendre qui fait quoi au sein du LESIM.

Surtout, j'ai eu la chance d'être traité dès le premier jour comme un membre à part entière de l'équipe, et non comme un simple "étudiant". Cette confiance m'a beaucoup aidé à sortir de ma réserve. Aujourd'hui, je me sens légitime pour prendre la parole lors des réunions, proposer mes propres idées, ou interagir avec d'autres ingénieurs du service pour résoudre un problème bloquant.

5.3 Difficultés rencontrées

5.3.1 La contrainte cross-platform

La difficulté technique qui m'a le plus marqué cette année est indéniablement la contrainte de portabilité Linux/Windows. Mon environnement de développement quotidien est Linux, et j'ai eu le réflexe naturel de tester mes solutions uniquement sur Linux dans un premier temps. L'échec du module `multiprocessing` sur Windows m'a obligé à repenser complètement mon approche et à systématiser les tests sur les deux plateformes. C'est une leçon que je n'aurais pas pu apprendre aussi concrètement en cours.

5.3.2 Prendre en main un code existant

Reprendre le PolyMeshPlugin après un stagiaire de cinq mois a été une expérience particulière. Le code fonctionnait conditionnellement, et certaines décisions d'architecture étaient difficiles à comprendre sans contexte. J'ai dû investir du temps pour lire, tester et comprendre l'existant avant de pouvoir contribuer efficacement. Cette phase, frustrante au début, m'a appris l'importance de la documentation et du code clair - une leçon que j'essaie d'appliquer dans mes propres contributions.

5.3.3 Équilibre école-entreprise

Ce passage d'un environnement professionnel à un environnement académique et inversement exige une vraie force mentale. À l'école, les projets sont très encadrés, souvent courts et évalués sur la note. En entreprise, les projets s'inscrivent dans le temps long, les problèmes n'ont pas de solution évidente trouvable sur internet, et l'objectif final est que le code soit robuste et maintenable au moins pour les dix prochaines années.

Bien que ce changement de rythme permanent soit parfois fatigant, il est extrêmement enrichissant. Chaque période faite à l'école me permet de revenir plus fort en entreprise, et chaque période d'entreprise m'apprend des choses que je ne pourrai jamais apprendre à l'école. Je vois désormais l'application directe de mes cours d'architecture logicielle ou d'algorithmique dans mon quotidien au CEA, et inversement, mon aisance sur Git et Linux m'a donné une grande avance pour mes TP à l'école.

Ce double rythme m'a également appris à mieux gérer mes priorités : savoir quand avancer seul, quand demander de l'aide, et comment reprendre un problème complexe après plusieurs jours d'interruption.

5.4 Perspectives pour les années 2 et 3

Cette première année a été, pour l'essentiel, une année de prise en main : comprendre l'environnement, apprendre les outils, contribuer à des plugins existants. Les bases sont maintenant posées.

En parallèle, j'aimerais apprendre le langage de programmation RUST avec des formations dispensées en interne. Je souhaite également consolider mes compétences en C++ scientifique, notamment en approfondissant ma maîtrise des outils de compilation CMake et des bibliothèques de calcul numérique. Le développement de **geogram2med** a été une première expérience stimulante dans ce domaine, mais je mesure encore l'étendue de ce qu'il me reste à apprendre.

Pour les deux années qui suivent, les perspectives techniques qui m'ont été présentées par mes encadrants sont ambitieuses, et porteront chacune sur le développement d'un nouveau plugin PyQt *from scratch* pour le module SMESH, de l'interface graphique jusqu'à l'intégration dans SALOME avec une API Python exposée — le même principe que ce que j'ai réalisé cette année.

La première mission, prévue en deuxième année, concernera l'intégration d'un algorithme de génération de couche prismatique issu de **code_saturne**, le solveur CFD open source d'EDF. Les couches prismatiques sont des éléments de maillage disposés en strates régulières au voisinage des parois, indispensables pour capturer précisément les gradients de vitesse et de température dans les simulations CFD. Leur intégration dans SALOME impliquera de concevoir un pipeline complet, de la lecture du maillage volumique d'entrée jusqu'à l'export du maillage enrichi au format **.med**, en passant par le couplage avec les bibliothèques de **code_saturne**. Cette mission a été identifiée comme point de départ car elle s'inscrit dans la continuité directe de ce que j'ai déjà réalisé, notamment la gestion de processus externes et la conversion de formats de maillage.

La seconde mission, plus complexe, portera sur l'intégration d'algorithmes de maillage hexaédrique dans un workflow multi-outils au sein de SMESH, en s'appuyant sur les travaux d'une thèse faite au LESIM en collaboration avec le LIHPC (Laboratoire en Informatique Haute Performance pour le Calcul et la Simulation). Contrairement au maillage tétraédrique, le maillage hexaédrique impose des contraintes topologiques strictes sur la géométrie d'entrée et nécessite des algorithmes de décomposition de domaine spécifiques — notamment une étape de *polycube labeling* permettant de paramétrer la géométrie avant génération du maillage. L'enjeu sera de coordonner plusieurs outils dans un workflow cohérent et de les exposer via une interface PyQt intégrée à SALOME.

Sur le plan personnel, je souhaite progresser sur plusieurs axes : approfondir ma compréhension de la mécanique des fluides numérique pour mieux comprendre les cas d'usage de mes développements, contribuer de manière plus autonome — c'est-à-dire identifier moi-même les problèmes et proposer des solutions sans attendre qu'on me les soumette — et m'impliquer davantage dans la communauté SALOME, notamment en participant aux discussions sur les évolutions de l'architecture du module SMESH.

Au-delà de l'aspect purement technique de l'écriture de code, mon objectif personnel ultime pour la fin de ce cycle est d'acquérir une véritable posture de chef de projet technique. J'aimerais être capable de partir d'un besoin utilisateur "brut", de rédiger moi-même les spécifications techniques, d'estimer le temps de développement nécessaire, de faire les bons choix d'architecture et de mener le développement de A à Z. Devenir ingénieur ne consistera pas seulement à savoir programmer, mais surtout à concevoir des solutions fiables et à les défendre dans un environnement industriel de haute exigence.

Conclusion

Cette première année d'apprentissage au sein du laboratoire LESIM du CEA Paris-Saclay a été bien plus qu'une introduction au monde professionnel : elle a représenté une confrontation directe entre les connaissances théoriques acquises en formation et les exigences concrètes d'un projet logiciel scientifique d'envergure internationale.

Sur le plan technique, mes contributions au MeshBooleanPlugin et au PolyMeshPlugin ont abouti à deux pull requests intégrées à la branche principale de SALOME, qui seront disponibles en open source dans la prochaine version publique de la plateforme. Ce résultat tangible, du code que j'ai écrit, relu, testé et défendu, et qui sera utilisé par des ingénieurs et chercheurs à travers le monde, est ce qui donne le plus de sens à cette première année.

Mais ce qui restera peut-être le plus formateur n'est pas une contribution technique précise. C'est d'avoir compris ce que signifie réellement écrire du code dans un contexte industriel : un code doit fonctionner sur Linux *et* Windows, que d'autres développeurs liront et feront évoluer dans dix ans, et dont la moindre régression peut impacter des simulations critiques. Cette exigence de rigueur, dans le test, dans la documentation, dans le choix d'architecture, est une culture que je n'aurais pas pu acquérir en environnement académique seul.

Le format de l'alternance lui-même s'est révélé plus structurant que je ne l'anticipais. Le passage régulier entre l'école et l'entreprise oblige à une discipline personnelle constante, mais offre en retour une double lecture permanente : les cours prennent un sens nouveau quand on en voit l'application directe, et les problèmes rencontrés en entreprise nourrissent une curiosité que les cours n'auraient pas suscitée.

Les deux années qui viennent s'annoncent plus ambitieuses, avec des missions portant sur des algorithmes de maillage hexaédrique et des couches prismatiques issues de `code_saturne`. Fort des bases acquises cette première année, je me sens prêt à aborder ces défis avec plus d'autonomie et de maturité, et à contribuer de manière plus proactive à l'évolution de SALOME.

Bibliographie

- [1] CEA & EDF, *SALOME Platform*, 2000. <https://www.salome-platform.org>
- [2] *MeshBooleanPlugin* – *SALOME Platform*, <https://github.com/SalomePlatform/MeshBooleanPlugin>.
- [3] *PolyMeshPlugin* – *SALOME Platform*, Dépôt interne CEA/EDF, sortie dans prochaine version de SALOME.
- [4] CEA & EDF, *SALOME Platform – GitHub*, <https://github.com/SalomePlatform>.
- [5] The CGAL Project, *CGAL – Computational Geometry Algorithms Library*, <https://www.cgal.org>.
- [6] G. Bernstein & D. Fussell, *CORK – A Fast, Robust Boolean Library*, <https://github.com/gilbo/cork>.
- [7] F. Trettner et al., *MCUT – A Library for Mesh Cutting*, <https://github.com/cutdigital/mcut>.
- [8] A. Jacobson et al., *libigl – A Simple C++ Geometry Processing Library*, <https://libigl.github.io>.
- [9] C. Blümel et al., *IRMB – Boolean Operations on Meshes*, <https://github.com/gcherchi/InteractiveAndRobustMeshBooleans>.
- [10] Kitware Inc., *VTK – The Visualization Toolkit*, <https://vtk.org>.
- [11] Symscape, *Polyhedral, Tetrahedral, and Hexahedral Mesh Comparison*, 2013. <https://www.symscape.com/polyhedral-tetrahedral-hexahedral-mesh-comparison>
- [12] CEA-List, *Le numérique au service du développement durable*, 2024. <https://list.cea.fr/fr/le-numerique-au-service-dun-developpement-plus-durable/>
- [13] ARCEP & ADEME, *Évaluation de l’empreinte environnementale du numérique en France*, 2023. https://www.arcep.fr/uploads/tx_gspublication/dossier-presse-Etude-Ademe-Arcep-lot3_mars2023.pdf
- [14] CEA – Direction des Systèmes d’Information (DSI), *La DSI et le numérique responsable*, Document interne CEA.
- [15] B. Lévy, *Geogram : A Programming Library of Geometric Algorithms*, Inria, 2012. <https://github.com/BrunoLevy/geogram>
- [16] RWTH Aachen University, *OpenVolumeMesh : A generic and polyhedral mesh data structure*. <https://www.graphics.rwth-aachen.de/software/openvolumemesh/>
- [17] Spatial (3DS), *MG-Cleaner – Mesh Cleaning Software*, <https://www.spatial.com/products/3d-precise-mesh>.

Lexique

1. **SALOME** — Simulation numérique par Architecture Logicielle en Open source et à Méthodologie d'Evolution environnement logiciel.
2. **CAO** (Conception Assistée par Ordinateur) — Modélisation géométrique 3D d'objets par ordinateur, en amont de la simulation.
3. **Tomographie** — Technique d'imagerie produisant des coupes 3D d'un objet convertibles en maillages.
4. **API** (Application Programming Interface) — Interface permettant à un programme d'interagir avec un autre de manière structurée.
5. **Pull request** — Demande de fusion soumise par un développeur sur une plateforme de gestion de code source (ici GitHub), proposant l'intégration de ses modifications dans la branche principale d'un dépôt. Elle est soumise à une revue par d'autres membres de l'équipe avant d'être acceptée (*mergée*) ou rejetée.
6. **Pylint** — Outil d'analyse statique de code pour le langage **Python**. Il vérifie le respect des conventions de style (norme **PEP 8**), détecte les erreurs de programmation et suggère des améliorations pour garantir la qualité et la maintenabilité du code.
7. **MG-Cleaner** — Outil sous licence commerciale de 3DS Spatial. Son utilisation nécessite également l'application d'algorithmes supplémentaires pour déplacer des nœuds sur les surfaces de la matrice, faciliter les opérations booléennes et assurer la fermeture des surfaces afin de définir un maillage volumique.
8. **CFD** (Computational Fluid Dynamics) — Simulation numérique des écoulements de fluides par résolution des équations de la mécanique des fluides.
9. **ASCII** (American Standard Code for Information Interchange) — Norme de codage de caractères représentant du texte informatique, utilisée pour les formats de fichiers lisibles par l'homme (par opposition aux formats binaires).
10. **Inria** (Institut National de Recherche en Informatique et en Automatique) — Organisme public de recherche français spécialisé en sciences et technologies du numérique. C'est au sein de cet institut qu'a été développée la bibliothèque Geogram utilisée pour le maillage.
11. **format .med** — Format de fichier standard du CEA pour l'échange de maillages et champs de simulation.
12. **Geogram** — Bibliothèque C++ de géométrie computationnelle utilisée pour générer des maillages polyédriques.
13. **DDRS** (Développement Durable et Responsabilité Sociétale) — Engagements d'un organisme intégrant les dimensions environnementales, sociales et économiques à ses activités.
14. **HPC** (High Performance Computing) — Calcul intensif mobilisant des clusters de serveurs pour résoudre des problèmes numériques complexes.